

B.TECH. (CSE)
SIX SEMESTER
Subject: Software Project Management
Subject Code: CUCS-342

MCQ

(a) A software comprises of :

- (i) Programs + Hardware + Software
- (ii) Programs + Operating procedures + Documentation manuals
- (iii) Hardware + Software
- (iv) None of the above

The correct answer is:

(ii) Programs + Operating procedures + Documentation manuals

(b) The most important task in a software project is

- (i) Implementation
- (ii) Controlling
- (iii) Closing
- (iv) Planning

The correct answer is:

(iv) Planning

(c) The critical path is calculated by

- (i) Making a forward pass
- (ii) Making a backward pass
- (iii) Both (a) and (b)
- (iv) None of the above

The correct answer is:

(iii) Both (a) and (b)

(d) The best paradigm for software engineering is

- (i) Waterfall model
- (ii) Prototyping
- (iii) Spiral model
- (iv) Combination of (a), (b), ©

The correct answer is:

(iv) Combination of (a), (b), (c)

(e) The schedule variance (SV) is given by

- (i) $SV = EV + PV$
- (ii) $SV = EV - PV$
- (iii) $SV = EV / PV$
- (iv) None of the above

The correct answer is:

(ii) $SV = EV - PV$

(f) Schedule Performance Index (SPI) is given by

- (i) $SPI = EV/AC$
- (ii) $SPI = EV/PV$
- (iii) $SPI = EV/CPI$
- (iv) None of the above

The correct answer is:

(ii) $SPI = EV / PV$

(g) Programmers make mistake during coding. These mistakes are known as

- (i) Failure
- (ii) Defects
- (iii) Bugs
- (iv) Errors

The correct answer is:

(iv) Errors

(h) CMM is a technique to

- (i) Improve software process
- (ii) Test the software
- (iii) Develop the software
- (iv) None of the above

The correct answer is:

(i) Improve software process

(i) Software quality is defined as

- (i) Conformance to user-specified requirements
- (ii) Achievement of extremely low defect rates
- (iii) Production of software with high reliability
- (iv) None of the above

The correct answer is:

(i) Conformance to user-specified requirements

(j) SQA stands for

- (i) Software quality added
- (ii) System quality audit
- (iii) Software quality assurance
- (iv) None of the above

The correct answer is:

(iii) Software Quality Assurance

(k) The SDLC consists of :

- (v) Analysis, design, coding
- (vi) Analysis, design, coding and testing
- (vii) Analysis, design, coding, testing and maintenance
- (viii) None of the above

The correct answer is:

(vii) Analysis, design, coding, testing and maintenance

(l) In addition to cost and time which other parameter is important for any project:

- (v) Quality
- (vi) Efficiency
- (vii) Speed
- (viii) None of the above

The correct answer is:

(v) Quality

(m) The time necessary to complete a project is referred to as

- (v) Implementation time
- (vi) Life time
- (vii) Operation cycle
- (viii) None of the above

The correct answer is:

(viii) None of the above

(n) If two activities are taken up at the same time, they are called as

- (v) Processor activities
- (vi) Successor activities
- (vii) Concurrent activities
- (viii) Periodical activities

The correct answer is:

(vii) Concurrent activities

(o) The cost variance (CV) is given by

- (v) $CV = EV - AC$
- (vi) $CV = EV + AC$
- (vii) $CV = EV/AC$
- (viii) None of the above

So, the correct option is:

(v) $CV = EV - AC$

(p) A planning meeting is also known as

- (v) Informal review
- (vi) Formal review

- (vii) Technical review
- (viii) Dynamic review

So, the correct answer is:

(v) Informal review

(q) Software testing is equal to

- (v) Software verification + Software Validation
- (vi) Software verification - Software Validation
- (vii) QA + QC
- (viii) None of the above

So, the correct option is:

(v) Software verification + Software Validation

(r) CMM stands for

- (v) Capacity measuring model
- (vi) Capacity memory model
- (vii) Capacity maturity model
- (viii) None of the above

(viii) None of the above

(s) SCM stands for

- (v) System control manager
- (vi) Software configuration management
- (vii) Software control management
- (iv) None of the above

So, the correct option is:

(vi) Software configuration management

(t) CBA stands for

- (v) Cost before assurance
- (vi) Common base assurance
- (vii) Cost benefit assurance
- (viii) Cost benefit analysis

So, the correct option is:

(viii) Cost benefit analysis

1. Write the characteristics of a project.

In **software project management**, the characteristics of a project remain similar but with some specialized aspects. Here's how they apply:

1. **Objective-Driven Development** – A software project aims to develop a specific software system or application.
2. **Defined Scope and Requirements** – Clear functionality and features must be outlined, often through Software Requirement Specifications (SRS).
3. **Software Development Lifecycle (SDLC)** – Projects follow structured phases like planning, design, development, testing, deployment, and maintenance.
4. **Temporary but Iterative** – While a project is temporary, software development often involves iterations or agile sprints.
5. **Team Collaboration** – Requires coordinated efforts among developers, testers, UI/UX designers, project managers, and stakeholders.
6. **Resource and Time Constraints** – Must be executed within budget, deadlines, and available infrastructure.
7. **Risk Management** – Includes risks like scope creep, security vulnerabilities, and integration challenges.
8. **Customer-Centric Development** – Stakeholder feedback and continuous iterations ensure usability and efficiency.
9. **Quality Assurance & Testing** – Rigorous software testing is essential for functionality, performance, and security.
10. **Maintenance and Support** – Even after project completion, software requires updates, bug fixes, and improvements.

2. What is software project estimation?

Software project estimation is the process of predicting the effort, cost, time, and resources required to complete a software project. It helps project managers plan effectively, allocate resources, and manage risks.

Here are key aspects of software project estimation:

1. **Effort Estimation** – Calculates the total work hours or manpower required.
2. **Time Estimation** – Determines the duration to complete the project.
3. **Cost Estimation** – Estimates the financial resources needed, including salaries, tools, and infrastructure.
4. **Resource Allocation** – Assesses hardware, software, and personnel requirements.
5. **Risk Consideration** – Accounts for uncertainties that may affect project success.

Common Estimation Techniques:

- **Expert Judgment** – Relies on experienced professionals to provide estimates based on past projects.
- **Analogous Estimation** – Uses historical data from similar projects for predictions.
- **Function Point Analysis (FPA)** – Estimates based on the complexity of software functionalities.
- **COCOMO (Constructive Cost Model)** – A formula-based estimation model that considers project size and complexity.
- **Use Case Points** – Estimates based on the number and complexity of use cases in the software.
- **Agile Estimation** – Includes techniques like Planning Poker and Story Points for iterative projects.

3. What are the types of project plan?

1. Master Project Plan

- This is the **primary document** that outlines the entire project, including objectives, timelines, budgets, risks, and stakeholders.
- It serves as a roadmap to ensure the software development stays on track.

2. Scope Management Plan

- Defines the project's **boundaries**, deliverables, and exclusions.
- Prevents **scope creep**, where additional features are added without proper assessment.

3. Schedule Management Plan

- **Breaks down** tasks into a timeline, using tools like Gantt charts and PERT diagrams.
- Defines milestones, deadlines, and dependencies to ensure timely delivery.

4. Cost Management Plan

- Estimates **budget requirements** by considering salaries, infrastructure, licenses, and unexpected expenses.
- Helps track costs and prevent budget overruns.

5. Quality Management Plan

- Sets **standards for testing** and validation to ensure the software functions correctly.
- Defines metrics like performance, security, and usability.

6. Resource Management Plan

- Lists **human resources**, tools, and technologies needed for development.
- Helps in hiring personnel and acquiring necessary software/hardware.

7. Risk Management Plan

- Identifies potential **risks**, such as delays, security vulnerabilities, and integration challenges.
- Defines **mitigation strategies** to minimize impact.

8. Communication Management Plan

- Establishes **how stakeholders will be informed** about project status.
- Includes meeting schedules, reports, and feedback channels.

9. Procurement Management Plan

- Governs **outsourcing and third-party acquisitions**, such as cloud services or external tools.
- Defines contracts and vendor management.

10. Change Management Plan

- Outlines **procedures for modifications**, whether in scope, features, or requirements.
- Helps control unexpected changes without disrupting project flow.

4. Explain PERT chart with example.

A **PERT (Program Evaluation and Review Technique) chart** is a project management tool used to visualize and schedule tasks, showing dependencies and timelines. It helps teams estimate the time required to complete tasks and identify the **critical path**—the sequence of tasks that determines the overall project duration.

Key Features of a PERT Chart:

1. **Nodes (Circles or Rectangles)** – Represent tasks or milestones.
2. **Arrows** – Show dependencies between tasks.
3. **Time Estimates** – Includes optimistic, pessimistic, and most likely durations for tasks.

4. **Critical Path** – The longest sequence of dependent tasks determining the shortest possible completion time.
5. **Slack Time** – Indicates how much flexibility a task has before it delays the project.

5. Write the difference between PERT and CPM.

PERT (Program Evaluation and Review Technique) and **CPM (Critical Path Method)** are both project management tools used for scheduling, planning, and controlling project activities. However, they differ in approach and application.

Feature	PERT (Program Evaluation and Review Technique)	CPM (Critical Path Method)
Focus	Deals with time estimation and project uncertainty	Focuses on time and cost optimization
Nature	Probabilistic (estimates time using optimistic, pessimistic, and most likely durations)	Deterministic (assumes fixed activity durations)
Application	Used for research, development, and complex projects with high uncertainty	Suitable for construction, engineering, and routine projects
Task Types	Used for non-repetitive tasks (projects with unique activities)	Used for repetitive tasks (standardized activities)
Time Estimates	Uses three time estimates: optimistic, pessimistic, and most likely	Uses single fixed time estimates per task
Critical Path	Focuses on event-based sequencing	Focuses on activity-based sequencing
Cost Analysis	Does not focus on cost directly	Includes cost control and optimization

6. What is pair programming?

Pair programming is an agile software development technique where two programmers work together at one workstation. One is the **"Driver"**, who writes the code, while the other is the **"Navigator"**, who reviews and provides real-time feedback. They frequently switch roles to enhance collaboration and problem-solving.

Key Benefits of Pair Programming:

1. **Improved Code Quality** – Bugs and errors are caught early through continuous review.
2. **Knowledge Sharing** – Helps junior developers learn from experienced programmers.
3. **Enhanced Problem-Solving** – Brainstorming leads to more optimized solutions.
4. **Better Team Collaboration** – Encourages communication and teamwork.
5. **Higher Productivity** – Reduces debugging time and accelerates development.

7. Explain software reviews.

Software Reviews

Software reviews are **formal or informal evaluations** of software artifacts, such as code, design documents, test plans, and requirements. They help improve quality, detect defects early, and ensure compliance with standards.

Types of Software Reviews

1. Peer Review (Informal Code Review)

- Developers review each other's code before merging changes.
- Helps maintain coding standards and reduce errors.

2. Technical Review

- Focuses on software design, algorithms, and architecture.
- Conducted by experts to ensure feasibility and optimization.

3. Code Review

- Examination of source code for readability, efficiency, and security.
- Can be **manual** (developer-led) or **automated** (using tools like SonarQube).

4. Walkthrough

- A semi-formal review where the author **explains** code or documentation.
- Helps in knowledge sharing and validating design decisions.

5. Inspection

- A formal review process **with structured defect detection**.
- Involves a moderator, reviewer, and author.

6. Audit

- Conducted by external teams to **check compliance** with industry regulations (ISO, GDPR).
- Ensures security and legal adherence.

8. What are the earned value indications?

Earned Value (EV) indicators are metrics used in project management to assess **performance, progress, and cost efficiency**. They help compare planned vs. actual project performance, ensuring timely delivery and budget control.

Key Earned Value Indicators:

1. **Planned Value (PV)**
 - The approved budget for scheduled work.
 - Represents expected costs **at a given point in time**.
2. **Earned Value (EV)**
 - The value of work **actually completed** versus planned.
 - Helps determine if the project is ahead or behind schedule.
3. **Actual Cost (AC)**

- The real expenses incurred for completed work.
- Used to evaluate cost efficiency.

Performance Metrics:

4. **Cost Variance (CV)** = $EV - AC$
 - If $CV > 0$ → The project is under budget.
 - If $CV < 0$ → The project is over budget.
5. **Schedule Variance (SV)** = $EV - PV$
 - If $SV > 0$ → The project is ahead of schedule.
 - If $SV < 0$ → The project is behind schedule.
6. **Cost Performance Index (CPI)** = EV / AC
 - If $CPI > 1$ → The project is cost-efficient.
 - If $CPI < 1$ → The project is cost-inefficient.
7. **Schedule Performance Index (SPI)** = EV / PV
 - If $SPI > 1$ → The project is ahead of schedule.
 - If $SPI < 1$ → The project is delayed.

9. What is software testing?

Software Testing

Software testing is the **process of evaluating a software application** to ensure it functions correctly, meets requirements, and is free of defects. It involves executing the software under controlled conditions and checking for bugs, security vulnerabilities, and performance issues.

Key Objectives of Software Testing:

1. **Identify Bugs & Defects** – Ensures the software is reliable and error-free.
2. **Validate Functionality** – Confirms the software meets user expectations.
3. **Enhance Performance & Security** – Helps optimize speed, security, and resource usage.
4. **Ensure Compliance** – Verifies adherence to industry standards and regulations.
5. **Improve Software Quality** – Guarantees a seamless user experience.

Types of Software Testing:

1. Manual Testing

- Conducted by human testers without automation.
- Example: **Exploratory Testing**, where testers simulate real-world user actions.

2. Automated Testing

- Uses scripts and tools to execute tests automatically.
- Example: **Selenium**, a popular automation framework.

3. Functional Testing

- Verifies the software's **features and functionality**.
- Example: **Unit Testing**, where individual components are tested separately.

4. Non-Functional Testing

- Focuses on **performance, security, usability, and scalability**.
- Example: **Load Testing**, which checks system behavior under heavy usage.

5. White-Box vs. Black-Box Testing

- **White-box testing:** Tests internal logic and code structure.
- **Black-box testing:** Evaluates functionality **without knowing the code**.

6. Regression Testing

- Ensures new changes **don't break existing features**.
- Example: After modifying a login module, testers verify that prior functionalities still work.

10. Write the difference between Quality assurance and Quality control.

Difference Between Quality Assurance (QA) and Quality Control (QC)

Quality Assurance (**QA**) and Quality Control (**QC**) are both essential for ensuring high standards in software development, but they serve different purposes.

Feature	Quality Assurance (QA)	Quality Control (QC)
Definition	A proactive process that ensures software development follows best practices and standards.	A reactive process that involves inspecting and testing the final product for defects.
Focus	Prevention – Ensures quality is built into development.	Detection – Identifies defects in the final product.
Activities	Process audits, documentation reviews, training, and compliance checks.	Software testing, inspections, debugging, and validation.
Responsibility	Performed by QA teams (process-oriented role).	Performed by Testers and QC teams (product-oriented role).
When Applied?	Before software development starts and throughout the lifecycle.	After development, during testing and before deployment.
Goal	To improve processes and prevent defects.	To ensure the final product meets quality standards and is defect-free.

11. Explain change request management.

Change Request Management

Change Request Management is the process of handling **modifications** in a project's scope, requirements, or execution plan. It ensures that changes are **properly evaluated, approved, and implemented** without disrupting the project's progress.

Key Aspects of Change Request Management

1. **Identification** – A change request is raised when stakeholders propose modifications (e.g., feature additions, requirement updates).
2. **Documentation** – The request is formally documented, detailing **why the change is needed** and its expected impact.

3. **Impact Analysis** – The project team evaluates the effect on **schedule, cost, resources, and risks**.
4. **Approval Process** – A Change Control Board (**CCB**) or project manager decides whether to approve, reject, or modify the request.
5. **Implementation** – If approved, the change is integrated into the development cycle with proper adjustments.
6. **Monitoring & Communication** – The change is tracked, tested, and communicated to relevant stakeholders.
7. **Closure** – Once successfully implemented, the change request is formally closed, ensuring proper documentation.

12. What are the CASE tools?

CASE (Computer-Aided Software Engineering) Tools

CASE tools are software applications designed to assist in **software development and project management** by automating tasks, improving accuracy, and enhancing productivity.

Types of CASE Tools

1. Upper CASE Tools (Front-End Design)

- Used in the **early stages** of development (planning & analysis).
- Examples: **ERD Tools**, Requirement Analysis Tools.

2. Lower CASE Tools (Back-End Development)

- Used in **later phases**, such as coding, testing, and maintenance.
- Examples: **Code Generators**, **Testing Tools**.

3. Integrated CASE Tools (End-to-End Solutions)

- Cover **both** front-end and back-end processes.
- Examples: **IBM Rational Rose**, **Enterprise Architect**.

4. Tools for Specific Development Phases

Phase	CASE Tool Examples
Requirement Analysis	JIRA, IBM DOORS
Design Modeling	UML Tools (StarUML, Visual Paradigm)
Coding & Development	Eclipse, NetBeans
Testing & Debugging	Selenium, JUnit
Maintenance & Version Control	Git, SVN

13. What is software project management?

Software Project Management

Software Project Management (SPM) is the **process of planning, organizing, executing, and monitoring** software development projects to ensure they are completed **on time, within budget, and according to requirements**. It involves coordinating teams, managing risks, and optimizing resources.

Key Responsibilities in Software Project Management:

1. **Project Planning** – Defining goals, scope, timelines, and deliverables.
2. **Resource Management** – Allocating human, technical, and financial resources efficiently.
3. **Risk Management** – Identifying potential issues and implementing mitigation strategies.
4. **Cost Estimation & Budgeting** – Calculating and controlling project expenses.
5. **Scheduling & Time Management** – Using tools like **Gantt charts** and **PERT diagrams** to track progress.
6. **Quality Assurance** – Ensuring that the software meets functional and non-functional requirements.
7. **Communication & Collaboration** – Maintaining clarity among stakeholders, developers, and testers.
8. **Change & Scope Management** – Handling modifications while minimizing disruptions.
9. **Testing & Deployment** – Verifying functionality, security, and reliability before release.
10. **Project Closure & Maintenance** – Completing documentation and ensuring ongoing support.

Common Methodologies Used in SPM:

- **Agile** – Iterative approach focusing on adaptability and customer feedback.
- **Waterfall** – Linear process where each phase is completed before the next begins.
- **Scrum** – A subset of Agile with short development cycles (**Sprints**) and daily stand-up meetings.
- **Kanban** – Uses visual boards to track task progress and workflow.

14. Explain project management cycle.

Project Management Cycle

The **Project Management Cycle** outlines the structured phases involved in planning, executing, and completing a project efficiently. It ensures that the project is delivered **on time, within budget, and meets its objectives**.

Phases of the Project Management Cycle:

1. Initiation Phase

- Defines the **purpose, goals, and feasibility** of the project.
- Identifies stakeholders and constraints.
- Key Document: **Project Charter**

2. Planning Phase

- Develops a detailed **project roadmap**, including scope, timeline, risks, and resource allocation.
- Tools: **Gantt charts, PERT diagrams, and Risk Assessment matrices**
- Key Activities:
 - Cost Estimation
 - Scheduling
 - Risk Management Strategy

3. Execution Phase

- The actual **implementation** of tasks begins, following the project plan.
- Teams collaborate to develop deliverables while ensuring quality standards.
- Uses Agile/Scrum frameworks, version control, and milestone tracking.

4. Monitoring & Controlling Phase

- Continuously **tracks progress** using KPIs (Key Performance Indicators).
- Adjusts scope, budget, and risks based on ongoing assessments.
- Tools: **Earned Value Analysis (EVA), Status Reports**

5. Closure Phase

- Finalizes the project with a formal **handover of deliverables**.
- Conducts project review meetings and documentation.
- Lessons learned are recorded for future improvements.

15. Write the characteristics of a project manager.

Characteristics of a Project Manager

A **Project Manager (PM)** plays a crucial role in planning, executing, and delivering successful projects while ensuring efficiency, collaboration, and problem-solving.

Key Characteristics of an Effective Project Manager

1. Leadership Skills

- Inspires and guides the team toward achieving project goals.
- Makes strategic decisions and motivates stakeholders.

2. Communication & Collaboration

- Clearly conveys project objectives, expectations, and updates.
- Bridges gaps between developers, clients, and executives.

3. Problem-Solving Ability

- Handles roadblocks efficiently and finds solutions proactively.
- Quickly adapts to changes and mitigates risks.

4. Time & Resource Management

- Ensures tasks stay **on schedule** and **within budget**.
- Optimizes team workload and resource allocation.

5. Risk Assessment & Decision Making

- Identifies potential risks early and develops mitigation strategies.
- Makes data-driven decisions to keep the project on track.

6. Technical Knowledge

- Understands relevant tools, methodologies, and software development processes.
- Familiar with Agile, Waterfall, and hybrid project management approaches.

7. Stakeholder & Client Management

- Maintains strong relationships with clients and stakeholders.
- Balances expectations and delivers value.

8. Conflict Resolution

- Effectively **manages disputes** within the team.

- Ensures a **collaborative work environment** without disruptions.

9. Adaptability & Innovation

- Embraces **new technologies** and industry trends.
- Adjusts plans dynamically when required.

10. Documentation & Reporting

- Ensures accurate project tracking, progress reporting, and compliance.
- Maintains structured documentation for future reference.

16. What is work break down structure?

Work Breakdown Structure (WBS)

A **Work Breakdown Structure (WBS)** is a **hierarchical decomposition** of a project into **smaller, manageable tasks**. It helps teams organize work efficiently, track progress, and estimate costs.

Key Features of WBS:

1. **Hierarchy-Based** – Breaks down the project into multiple levels.
2. **Defined Scope** – Clearly outlines deliverables and tasks.
3. **Task Dependencies** – Shows relationships between different activities.
4. **Resource Allocation** – Helps assign work efficiently.
5. **Measurable Outcomes** – Ensures each task contributes to project goals.

17. What are the methods of WBS representation?

Methods of Work Breakdown Structure (WBS) Representation

A **Work Breakdown Structure (WBS)** can be represented in multiple ways to visually organize and define project tasks. The three most common methods are:

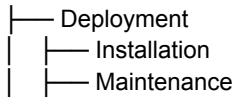
1. Hierarchical (Tree Structure) Representation

- The most commonly used format, displaying tasks in a **top-down, parent-child hierarchy**.
- It starts with the **main project** at the top, branching into subcomponents and further breakdowns.
- Useful for **showing dependencies** and the logical flow of tasks.

Example:

```

Software Development Project
├── Requirement Analysis
│   ├── Identify user needs
│   └── Select hardware/software
├── System Design
│   ├── Database architecture
│   └── UI/UX design
├── Development
│   ├── Frontend implementation
│   └── Backend coding
└── Testing
    ├── Unit testing
    └── Integration testing
  
```



2. Tabular (Outline) Representation

- Presents the WBS in a **structured table format**, listing components numerically.
- Often used in **formal documentation** where detailed descriptions of tasks are needed.

Example:

WBS Code	Task Name	Description
1.0	Software Development	Main project goal
1.1	Requirement Analysis	Define user needs, specifications
1.2	System Design	Database structure, UI/UX planning
1.3	Development	Coding frontend and backend
1.4	Testing	Debugging, validation checks
1.5	Deployment	Installation and system launch

3. Graphical (Flowchart or Gantt Chart) Representation

- Uses **flow diagrams** or **Gantt charts** for a more visual approach.
- Helps in **tracking timelines, dependencies, and milestones**.
- Ideal for **project planning tools like Microsoft Project or JIRA**.

18. What is Project Schedule?

Project Schedule

A **Project Schedule** is a detailed timeline that outlines the tasks, milestones, and dependencies within a project. It serves as a **roadmap** for tracking progress, ensuring timely completion, and managing resources efficiently.

Key Components of a Project Schedule:

1. **Task List** – Defines specific activities required to complete the project.
2. **Timeline & Deadlines** – Specifies start and end dates for each task.
3. **Dependencies** – Shows relationships between tasks (e.g., Task B starts after Task A is completed).
4. **Milestones** – Highlights significant points or achievements in the project lifecycle.
5. **Resource Allocation** – Assigns human, technical, and financial resources.
6. **Risk Management** – Plans for potential delays or unforeseen issues.
7. **Monitoring & Updates** – Ensures adjustments are made as needed during execution.

19. What are the dimensions of project monitoring and control?

Dimensions of Project Monitoring and Control

Project **Monitoring and Control** ensures that a project stays on track, meets objectives, and adapts to risks. It involves systematic tracking of project progress across multiple dimensions:

Key Dimensions:

1. Scope Monitoring

- Ensures that the project stays within its **defined boundaries**.
- Prevents **scope creep**, where extra tasks extend beyond the original plan.

2. Schedule Control

- Tracks **timelines and deadlines** using tools like **Gantt charts** and **PERT diagrams**.
- Identifies delays and adjusts schedules to meet milestones.

3. Cost Management

- Monitors the budget to ensure **cost efficiency**.
- Uses **Earned Value Analysis (EVA)** to compare planned vs. actual costs.

4. Quality Control

- Verifies that deliverables meet **required standards** through testing and audits.
- Applies **Quality Assurance (QA)** and **Quality Control (QC)** techniques.

5. Risk Management

- Identifies **potential risks** (technical, financial, organizational) early.
- Implements **mitigation strategies** to avoid project disruptions.

6. Resource Utilization

- Ensures **optimal allocation** of personnel, software, and hardware.
- Prevents bottlenecks caused by underutilization or overload.

7. Communication & Stakeholder Management

- Maintains transparency through **status reports, meetings, and updates**.
- Ensures alignment between **stakeholder expectations** and project execution.

8. Compliance & Documentation

- Ensures adherence to **legal and industry regulations** (e.g., GDPR, ISO standards).
- Maintains proper documentation for audits and future reference.

20. What is earned value analysis?

Earned Value Analysis (EVA) is a project management technique that provides a quantitative measure of project performance by comparing the work planned with the work actually completed, and the associated costs. EVA integrates scope, cost, and schedule into a single framework, allowing managers to assess how well the project is progressing.

Key Components of EVA:

1. **Planned Value (PV):**
 - Also known as the Budgeted Cost of Work Scheduled (BCWS).

- Represents the amount of work that should have been completed by a certain time, based on the project's baseline plan.
- 2. **Earned Value (EV):**
 - Also known as the Budgeted Cost of Work Performed (BCWP).
 - Measures the value of work that has actually been completed at a specific point in time, expressed in terms of the approved budget.
- 3. **Actual Cost (AC):**
 - Also known as the Actual Cost of Work Performed (ACWP).
 - Tracks the actual expenditure incurred for the work completed by that point.

Performance Metrics Derived from EVA:

- **Cost Variance (CV):**
 - **Formula:** $CV = EV - AC$
 - Indicates whether the project is under or over budget. A positive CV means the project is under budget, while a negative CV signifies cost overruns.
- **Schedule Variance (SV):**
 - **Formula:** $SV = EV - PV$
 - Reflects whether the project is ahead or behind schedule. A positive SV suggests the project is ahead of schedule, and a negative SV means it's behind.
- **Cost Performance Index (CPI):**
 - **Formula:** $CPI = EV / AC$
 - Measures cost efficiency. A CPI greater than 1 indicates the project is cost-efficient, while less than 1 shows inefficiencies.
- **Schedule Performance Index (SPI):**
 - **Formula:** $SPI = EV / PV$
 - Gauges schedule efficiency. Values above 1 mean better-than-planned performance, and values below 1 suggest delays.

21. Write the principle of testing.

Principles of Software Testing

Software testing is guided by several fundamental principles that help ensure effective defect detection, quality assurance, and reliable software performance. These principles improve test efficiency and reduce risks in development.

1. Testing Shows the Presence of Defects, Not Their Absence

- Testing can identify **existing bugs**, but it **cannot prove that a system is completely error-free**.
- Even after extensive testing, undiscovered defects may still exist.

2. Exhaustive Testing is Impossible

- It is **not feasible to test every possible input, scenario, or combination** due to complexity and cost.
- Instead, testing should prioritize **high-risk areas** based on business needs.

3. Early Testing Saves Time and Cost

- Detecting defects **early in the software development lifecycle** (SDLC) reduces rework and costs.
- Shifting testing to the early stages helps prevent major failures.

4. Defect Clustering

- A small number of modules typically contain the majority of defects.
- Focusing testing on **high-risk areas** increases efficiency in defect detection.

5. Pesticide Paradox

- Running the **same test cases repeatedly** eventually becomes ineffective.
- To find new defects, testers must **update and diversify test cases** regularly.

6. Testing is Context-Dependent

- Different applications require different testing approaches.
- A banking system needs **high security testing**, while a game app needs **performance testing**.

7. Absence of Errors is Not the Same as a Functioning System

- A software product can be **technically free of bugs** but still fail due to usability issues or unmet customer needs.
- Testing should focus on **business requirements and user satisfaction**

22. Write the difference between program verification and validation.

Feature	Program Verification	Program Validation
Definition	Ensures that the software meets design specifications and follows correct coding standards.	Confirms that the software meets user requirements and delivers expected outcomes.
Focus	Internal correctness of the software (logic, syntax, and structure).	External correctness (functionality and usability).
Objective	To check if the software is built correctly .	To ensure the right software is built .
Methodology	Uses formal methods like code reviews, inspections, static analysis .	Involves testing, user feedback, and real-world execution .
Timing	Performed during development (early phase).	Conducted after development , before deployment.
Examples	Checking for syntax errors, following coding standards, verifying correct algorithm implementation.	Running test cases, checking user expectations, performing usability testing.

Example Scenario:

If you're developing a **Face Recognition-Based Attendance System** (like your project):

- **Verification:** Ensures the **face recognition algorithm follows correct logic** and coding standards.
- **Validation:** Confirms that **the system correctly identifies faces and records attendance** based on real-world testing.

23. What are the SCM tasks?

Software Configuration Management (SCM) Tasks

Software Configuration Management (SCM) is the process of tracking and controlling changes in software development to ensure consistency, efficiency, and reliability.

Key SCM Tasks:

1. Configuration Identification

- Defines **what needs to be tracked**, including source code, requirements, test cases, and documentation.
- Establishes naming conventions and versioning guidelines.

2. Change Control & Management

- Ensures proper handling of modifications through **Change Requests (CRs)**.
- Uses a **Change Control Board (CCB)** to review and approve changes.

3. Version Control

- Maintains different software versions using **Git, SVN, or Mercurial**.
- Helps track changes, revert to previous versions, and prevent conflicts.

4. Configuration Auditing

- Conducts reviews to ensure that approved modifications are correctly implemented.
- Verifies compliance with standards and best practices.

5. Build Management

- Automates the **compilation and deployment** process using tools like **Maven, Gradle, or Jenkins**.
- Ensures consistency between different software builds.

6. Release Management

- Handles software releases, ensuring smooth deployment and rollback if needed.
- Creates **release notes** for tracking updates and improvements.

7. Software Baselining

- Defines stable reference points (**baselines**) in the development process for comparison and rollback.
- Examples include baselines for **requirements, design, and testing**.

24. What is cost benefit analysis?

Cost-Benefit Analysis (CBA) in Project Management

Cost-Benefit Analysis (CBA) is a systematic approach used to **evaluate the financial feasibility and economic value** of a project by comparing its total costs with its total benefits. It helps decision-makers determine whether a project is worth pursuing.

Key Components of Cost-Benefit Analysis

1. Costs (Expenditures)

- **Direct Costs** (e.g., labor, materials, software licenses)
- **Indirect Costs** (e.g., overhead, administrative expenses)
- **Opportunity Costs** (benefits lost by choosing one alternative over another)
- **Intangible Costs** (e.g., employee morale impact, brand reputation risk)

2. Benefits (Gains)

- **Tangible Benefits** (measurable, e.g., increased revenue, cost savings)
 - **Intangible Benefits** (hard to quantify, e.g., improved customer satisfaction, brand value)
 - **Direct & Indirect Benefits** (e.g., faster processing time leading to higher productivity)
-

Steps in Conducting a Cost-Benefit Analysis

1. **Define Project Scope & Objectives**
 - What problem is the project solving?
 - What are the expected outcomes?
 2. **Identify & Quantify Costs & Benefits**
 - List all expenses and gains over the project's lifecycle.
 - Assign monetary values where possible.
 3. **Compare Costs vs. Benefits**
 - **Net Benefit (NB) = Total Benefits – Total Costs**
 - **Benefit-Cost Ratio (BCR) = Total Benefits / Total Costs**
 - **BCR > 1** → Project is economically viable.
 - **BCR < 1** → Costs outweigh benefits (reconsider).
 4. **Consider Time Value of Money (Discounting)**
 - Future costs/benefits are discounted to **Present Value (PV)** using a discount rate.
 5. **Perform Sensitivity & Risk Analysis**
 - Test how changes in assumptions (e.g., cost overruns, lower benefits) affect results.
 6. **Make a Decision**
 - Proceed if benefits justify costs; else, explore alternatives.
-